IBC (Inter-Blockchain Communication) 분석 문서

개요

이 문서는 Mars 블록체인 애플리케이션에서 IBC (Inter-Blockchain Communication) 모듈의 설정과 초기화 과정을 상세히 분석합니다. IBC는 서로 다른 블록체인 간의 안전한 통신을 가능하게 하는 프로토콜입니다.

IBC 아키텍처 개요

1. IBC 핵심 구성 요소

```
IBC 스택 구조:

┌─────────────────────────────────────┐

│          애플리케이션 레이어          │

│   (Transfer, ICA, Custom Modules)   │

├─────────────────────────────────────┤

│               IBC 미들웨어           │

│       (라우팅, 인증, 순서 보장)       │

├─────────────────────────────────────┤

│               IBC 코어               │

│     (Client, Connection, Channel)   │

├─────────────────────────────────────┤
```

```
│              라이트 클라이언트            │
│      (Tendermint, Solo Machine)      │
└──────────────────────────────────────┘
```

2. IBC 모듈 구성

2.1 코어 IBC 모듈

- **IBCKeeper**: IBC 프로토콜의 핵심 기능 관리
- **ClientKeeper**: 다른 체인의 상태를 추적하는 라이트 클라이언트 관리
- **ConnectionKeeper**: 체인 간 연결 관리
- **ChannelKeeper**: 애플리케이션 간 채널 관리

2.2 IBC 애플리케이션 모듈

- **Transfer**: 토큰 전송 기능
- **ICA (Interchain Accounts)**: 체인 간 계정 제어
- Controller: 다른 체인의 계정을 제어
- Host: 다른 체인으로부터 제어받는 계정 관리

2.3 라이트 클라이언트 모듈

- **Tendermint**: Tendermint 합의 기반 체인과의 통신
- **Solo Machine**: 단일 서명자 기반 클라이언트

app.go에서의 IBC 설정

1. IBC Keeper 선언

```
// === IBC (Inter-Blockchain Communication) Keeper들 ===

// IBCKeeper는 IBC 프로토콜의 핵심 기능을 관리합니다

IBCKeeper *ibckeeper.Keeper



// ICAControllerKeeper는 Interchain Accounts의 컨트롤러 기능을 관리합니다

ICAControllerKeeper icacontrollerkeeper.Keeper



// ICAHostKeeper는 Interchain Accounts의 호스트 기능을 관리합니다

ICAHostKeeper icahostkeeper.Keeper



// TransferKeeper는 IBC를 통한 토큰 전송을 관리합니다

TransferKeeper ibctransferkeeper.Keeper
```

2. 의존성 주입에서 IBC 제외

IBC 모듈들은 아직 새로운 의존성 주입 시스템을 지원하지 않기 때문에, `depinject.Inject()` 호출에서 제외되고 별도로 수동 등록됩니다.

3. 수동 IBC 모듈 등록

```
// === 5단계: 레거시 모듈 등록 ===

// IBC 관련 모듈들은 아직 새로운 앱 와이어링을 지원하지 않으므로

// 수동으로 등록해야 합니다. 이는 ibc.go 파일에서 처리됩니다.

if err := app.registerIBCModules(appOpts); err != nil {
```

```
panic(err)
}
```

ibc.go에서의 상세 구현

1. 저장소 키 등록

```
// IBC 모듈들을 위한 저장소 키 등록
if err := app.RegisterStores(
storetypes.NewKVStoreKey(ibcexported.StoreKey), // IBC 코어
storetypes.NewKVStoreKey(ibctransfertypes.StoreKey), // Transfer
storetypes.NewKVStoreKey(icahosttypes.StoreKey), // ICA Host
storetypes.NewKVStoreKey(icacontrollertypes.StoreKey), // ICA Controller
); err != nil {
return err
}
```

2. 레거시 파라미터 서브스페이스 설정

```
// 레거시 파라미터 시스템을 위한 키 테이블 등록
keyTable := ibcclienttypes.ParamKeyTable()
keyTable.RegisterParamSet(&ibcconnectiontypes.Params{})
app.ParamsKeeper.Subspace(ibcexported.ModuleName).WithKeyTable(keyTable)
```

3. IBC Keeper 초기화

```
// IBC 코어 키퍼 생성
app.IBCKeeper = ibckeeper.NewKeeper(
app.appCodec, // 코덱
runtime.NewKVStoreService(app.GetKey(ibcexported.StoreKey)), // 저장소
app.GetSubspace(ibcexported.ModuleName), // 파라미터 서브스페이스
app.UpgradeKeeper, // 업그레이드 키퍼
govModuleAddr, // 거버넌스 권한 주소
)
```

4. IBC 애플리케이션 Keeper 초기화

4.1 Transfer Keeper

```
app.TransferKeeper = ibctransferkeeper.NewKeeper(
app.appCodec, // 코덱
runtime.NewKVStoreService(app.GetKey(ibctransfertypes.StoreKey)), // 저장소
app.GetSubspace(ibctransfertypes.ModuleName), // 파라미터
app.IBCKeeper.ChannelKeeper, // 채널 키퍼 (송신)
app.IBCKeeper.ChannelKeeper, // 채널 키퍼 (수신)
app.MsgServiceRouter(), // 메시지 라우터
app.AuthKeeper, // 계정 키퍼
app.BankKeeper, // 은행 키퍼
```

```
govModuleAddr, // 거버넌스 권한
)
```

4.2 ICA Host Keeper

```
app.ICAHostKeeper = icahostkeeper.NewKeeper(
app.appCodec, // 코덱
runtime.NewKVStoreService(app.GetKey(icahosttypes.StoreKey)), // 저장소
app.GetSubspace(icahosttypes.SubModuleName), // 파라미터
app.IBCKeeper.ChannelKeeper, // ICS4Wrapper
app.IBCKeeper.ChannelKeeper, // 채널 키퍼
app.AuthKeeper, // 계정 키퍼
app.MsgServiceRouter(), // 메시지 라우터
app.GRPCQueryRouter(), // 쿼리 라우터
govModuleAddr, // 거버넌스 권한
)
```

4.3 ICA Controller Keeper

```
app.ICAControllerKeeper = icacontrollerkeeper.NewKeeper(
app.appCodec, // 코덱
runtime.NewKVStoreService(app.GetKey(icacontrollertypes.StoreKey)), // 저장소
app.GetSubspace(icacontrollertypes.SubModuleName), // 파라미터
app.IBCKeeper.ChannelKeeper, // 채널 키퍼 (송신)
```

```
app.IBCKeeper.ChannelKeeper, // 채널 키퍼 (수신)
app.MsgServiceRouter(), // 메시지 라우터
govModuleAddr, // 거버넌스 권한
)
```

5. IBC 모듈 스택 구성

```
// IBC 모듈 스택을 하위에서 상위 순서로 생성
var (
transferStack porttypes.IBCModule = ibctransfer.NewIBCModule(app.TransferKeeper)
transferStackV2 ibcapi.IBCModule = ibctransferv2.NewIBCModule(app.TransferKeeper
icaControllerStack porttypes.IBCModule = icacontroller.NewIBCMiddleware(app.ICAC
icaHostStack porttypes.IBCModule = icahost.NewIBCModule(app.ICAHostKeeper)
)
```

6. IBC 라우터 설정

6.1 IBC v1 라우터

```
ibcRouter := porttypes.NewRouter().
AddRoute(ibctransfertypes.ModuleName, transferStack). // Transfer 모듈
AddRoute(icacontrollertypes.SubModuleName, icaControllerStack). // ICA Controlle
AddRoute(icahosttypes.SubModuleName, icaHostStack) // ICA Host
```

6.2 IBC v2 라우터

```
ibcv2Router := ibcapi.NewRouter().
AddRoute(ibctransfertypes.PortID, transferStackV2) // Transfer v2
```

7. 라이트 클라이언트 등록

```
clientKeeper := app.IBCKeeper.ClientKeeper
storeProvider := clientKeeper.GetStoreProvider()

// Tendermint 라이트 클라이언트
tmLightClientModule := ibctm.NewLightClientModule(app.appCodec, storeProvider)
clientKeeper.AddRoute(ibctm.ModuleName, &tmLightClientModule)

// Solo Machine 라이트 클라이언트
soloLightClientModule := solomachine.NewLightClientModule(app.appCodec, storePro
clientKeeper.AddRoute(solomachine.ModuleName, &soloLightClientModule)
```

8. 모듈 등록

```
if err := app.RegisterModules(
ibc.NewAppModule(app.IBCKeeper), // IBC 코어
ibctransfer.NewAppModule(app.TransferKeeper), // Transfer
```

```
icamodule.NewAppModule(&app.ICAControllerKeeper, &app.ICAHostKeeper), // ICA

ibctm.NewAppModule(tmLightClientModule), // Tendermint 클라이언트

solomachine.NewAppModule(soloLightClientModule), // Solo Machine 클라이언트

); err != nil {

return err

}
```

크로스체인 통신 아키텍처

1. 통신 흐름

```
    체인 A                          체인 B

┌──────────────┐                ┌──────────────┐

| Application |◄──────────────┤ Application |

├──────────────┤                ├──────────────┤

| IBC Modul e |◄──────────────┤ IBC Module |

├──────────────┤                ├──────────────┤

|   IBC Core   |◄──────────────┤   IBC Core   |

├──────────────┤                ├──────────────┤

|Light Client |◄──────────────┤Light Client |

└──────────────┘                └──────────────┘
```

2. 연결 설정 과정

1. **클라이언트 생성**: 각 체인이 상대방 체인의 라이트 클라이언트 생성
2. **연결 핸드셰이크**: 체인 간 연결 설정 (Connection)
3. **채널 핸드셰이크**: 애플리케이션 간 채널 설정 (Channel)
4. **패킷 전송**: 실제 데이터 패킷 전송

3. 보안 모델

- **라이트 클라이언트**: 상대방 체인의 상태 검증
- **머클 증명**: 상태 변경의 암호학적 증명
- **타임아웃**: 패킷 전송 시간 제한
- **순서 보장**: 패킷 순서 및 중복 방지

주요 특징

1. 모듈성

- 각 IBC 애플리케이션은 독립적으로 개발 가능
- 표준 IBC 인터페이스를 통한 상호 운용성

2. 확장성

- 새로운 라이트 클라이언트 추가 가능
- 커스텀 IBC 애플리케이션 개발 가능

3. 보안성

- 암호학적 증명을 통한 상태 검증
- 각 체인의 보안 모델 유지

4. 호환성

- 다양한 합의 알고리즘 지원
- 레거시 시스템과의 호환성 유지

개발 고려사항

1. 성능

- 라이트 클라이언트 업데이트 비용
- 패킷 전송 지연시간
- 저장소 사용량 최적화

2. 보안

- 라이트 클라이언트 공격 방어
- 타임아웃 설정 최적화
- 권한 관리 체계

3. 운영

- 릴레이어 운영 및 모니터링
- 업그레이드 호환성 관리
- 장애 복구 절차